

Laporan Simulasi Robot Sepakbola

Author(s): Aisyah Ayu Fitriani
Teacher: Muhammad Qomaruz Zaman, S.T., M.T., Ph.D.
Class name: Sistem Robot Otonom
YouTube video link: <https://youtu.be/WxBTJHcZ4Bk>
GitHub link: https://github.com/Asyah24/Tugas_SR0_Sepakbola

1 Pendahuluan

Pada tugas ini digunakan simulasi pertandingan sepakbola sederhana yang dijalankan di dalam lingkungan CoppeliaSim dengan antarmuka ZMQ Remote API. Terdapat tiga robot otonom yang berperan masing-masing sebagai *Striker*, *Goalkeeper*, dan *Participant* (pemain patroli). Kode python `robot_sepakbola_v4.py` mengatur perilaku seluruh agen tersebut secara bersamaan dalam satu loop simulasi.

2 Tujuan

1. Mengimplementasikan sistem multi-robot otonom pada simulasi CoppeliaSim menggunakan Python dan ZMQ Remote API.
2. Menerapkan mesin keadaan (*finite state machine*) pada robot *Striker* agar mampu merencanakan dan mengeksekusi tendangan ke gawang lawan secara otomatis.
3. Mengimplementasikan perilaku penjaga gawang (*Goalkeeper*) yang reaktif terhadap posisi bola.
4. Menerapkan perilaku patroli sederhana pada robot *Participant* sebagai penghalang dinamis di lapangan.
5. Mengelola siklus reset otomatis permainan menggunakan komponen *BallMonitor* berbasis kecepatan dan jarak bola.

3 Struktur dan Fungsi Kode

3.1 Konstanta Global

Bagian awal kode mendefinisikan seluruh konstanta yang digunakan sepanjang simulasi, antara lain nama objek di CoppeliaSim, parameter kecepatan motor, dan parameter geometri lapangan.

```
1 OBJ_BALL = "/Bola_Merah"
2 OBJ_ROBOT_GK = "/Robot_Pemain"
3 OBJ_ROBOT_ST = "/Robot_Lawan_01"
4 OBJ_ROBOT_PR = "/Robot_Lawan_02"
5 OBJ_GOAL = "/Gwang_Kuning"
6 NORMAL_SPEED = 4.0
7 SPRINT_SPEED = 12.0
8 KICK_VEL = 4.5
9 SETUP_DIST = 1.00
10 STOP_OFFSET = 0.50
11 KICK_RADIUS = 0.20
12 GOAL_WIDTH_FB = 0.80
13 GK_DEF_DIST = 0.50
14 GK_SLIDE_MAX = 0.65
15 FIELD_Y_MIN = -1.20
16 FIELD_Y_MAX = 1.20
17 RESET_DELAY = 7.0
18 BALL_MOVE_THRESH = 0.05
19 BALL_STILL_DIST = 0.10
```

3.2 Kelas Bot — Kelas Dasar Robot

Kelas Bot adalah kelas induk yang diwarisi oleh seluruh agen robot. Kelas ini menyediakan antarmuka dasar untuk membaca posisi dan orientasi robot, mengontrol motor roda, serta melakukan navigasi menuju titik tujuan. Fungsi kunci pada kelas ini adalah `go()`, yang mengimplementasikan kendali gerak proportional.

```
1 def go(self, tx, ty, spd=NORMAL_SPEED, thr=0.08):
2     p = self.pos()
3     dx, dy = tx - p[0], ty - p[1]
4     dist = math.sqrt(dx*dx + dy*dy)
5     if dist < thr:
6         self.stop()
7         return True
8     target_yaw = math.atan2(dy, dx)
9     ang_err = norm_ang(target_yaw - self.yaw())
10    v_ang = clamp(5.0 * ang_err, -spd, spd)
11    if abs(ang_err) > 0.5:
12        self.drive(-v_ang * 0.6, v_ang * 0.6)
13    else:
14        v_lin = spd * clamp(dist / 0.30, 0.1, 1.0)
15        v_lin *= (1.0 - abs(ang_err) / 1.5)
16        v_lin = clamp(v_lin, 0.3, spd)
17        self.drive(clamp(v_lin - v_ang, -spd, spd),
18                  clamp(v_lin + v_ang, -spd, spd))
19    return False
```

3.3 Kelas Goalkeeper — Penjaga Gawang

Kelas Goalkeeper mewarisi Bot dan bertugas mempertahankan gawang. Robot menghitung posisi Y ideal berdasarkan posisi bola dengan faktor pembobot 0.4 agar tidak terlalu agresif.

```
1 def update(self, ball_pos):
2     bx, by = ball_pos[0], ball_pos[1]
3     ideal_y = self.goal_cy + 0.40 * (by - self.goal_cy)
4     ideal_y = clamp(ideal_y, -GK_SLIDE_MAX, GK_SLIDE_MAX)
5     p = self.pos()
6     dx = self.def_x - p[0]
7     dy = ideal_y - p[1]
8     dist = math.sqrt(dx*dx + dy*dy)
9     if dist > 0.06:
10        self.go(self.def_x, ideal_y, spd=SPRINT_SPEED, thr=0.06)
11    else:
12        self.face_only(bx, by, thr=0.10)
```

3.4 Kelas Striker — Penyerang

Kelas Striker menggunakan *finite state machine* (FSM) dengan empat keadaan utama:

State	Deskripsi
GOTO SETUP	Robot bergerak ke titik anjang-angang di belakang bola.
FACE TARGET	Robot berhenti dan memutar badan menghadap titik target gawang.
CHARGE KICK	Robot berlari kencang menuju bola dan melakukan tendangan.
WAIT RESET	Robot menunggu BallMonitor memicu reset siklus berikutnya.

3.5 Kelas BallMonitor — Pemantau Bola

BallMonitor bertugas mendeteksi pergerakan bola dan memicu reset setelah `RESET_DELAY` tercapai.

```
1 def update(self, ball_pos):
2     spd = self._ball_velocity(ball_pos)
3     dist_init = d2(ball_pos[:2], self.ball_init[:2])
4     if self.state == self.STILL:
5         if spd > BALL_MOVE_THRESH and dist_init > BALL_STILL_DIST:
```

```

6         self.state = self.MOVING
7         self.move_time = time.time()
8     elif self.state == self.MOVING:
9         elapsed = time.time() - self.move_time
10        if elapsed >= RESET_DELAY:
11            self.state = self.WAITING
12            return True # trigger reset
13    return False

```

3.6 Kelas Participant — Robot Patroli

Participant bergerak bolak-balik sepanjang sumbu Y di antara posisi awal bola dan gawang untuk menciptakan penghalang dinamis yang realistis.

3.7 Fungsi main()

Fungsi utama menginisialisasi koneksi, membuat agen, dan menjalankan loop simulasi pada laju 62.5 Hz.

4 Alur Kerja Keseluruhan Sistem

1. **Inisialisasi:** Koneksi dibuka, posisi bola/gawang dibaca, agen dibuat.
2. **Loop Simulasi:** *BallMonitor* memantau bola, *Striker* eksekusi FSM, *Goalkeeper* bertahan, dan *Participant* patroli.
3. **Reset Siklus:** Setelah RESET_DELAY, bola dan robot dikembalikan ke posisi awal untuk siklus baru.

5 Kesimpulan

Kode `robot_sepakbola_v4.py` berhasil mengimplementasikan sistem multi-robot otonom berbasis FSM dan kendali proporsional. Arsitektur modular ini memudahkan pengembangan strategi permainan yang lebih kompleks di masa depan.